

西安电子科技大学网信院

编译原理综合实验

实验报告

——实现绘图语言解释器

班级　　2318039

姓名　　魏朝旭

学号　　23009201082

日期　　2024.12.27

目录

1 总体任务

2 函数绘图语言简介

3 函数绘图程序举例

4 开发环境及配置

5 基本原理与解决思路

6 关键类及主要方法

7 测试截图

8 总体体会（遇到的问题与体会）

1 总体任务

本项目的核心目标是开发一个解释器，该解释器能够接受并处理一种特定绘图语言编写的源程序。通过语法和语义分析，解释器将直接在显示屏上绘制出源程序所规定的图形。本项目将具体任务划分为以下几个部分：

1) 设计并实现一个词法分析器

2) 设计并实现一个语法分析器

3) 实现语法制导的翻译函数，用于图形绘制

撰写实验报告，总结分析实验过程与结果

2 函数绘图语言简介

1) 本绘图语言支持以下功能：

1、 循环绘图（FOR-DRAW）：通过循环结构绘制图形。

2、 比例设置（SCALE）：调整图形的横纵坐标比例。

3、 角度旋转（ROT）：对图形进行旋转变换。

4、 坐标平移（ORIGIN）：改变图形的原点位置。

5、 注释（ -- 或 // ）：使用//或--添加注释，忽略注释后的内容。

2) 屏幕（窗口）的坐标系

1、 左上角为原点

2、 X 方向从左到右增长

3、 Y 方向从上到下增长（与一般坐标系相反）

3) 函数绘图语言满足下述规定（原则）：

1、各类语句可以按任意次序书写，且语句以分号结尾。源程序中的语句以它们出现的先后顺序处理。

2、ORIGIN、ROT 和 SCALE 语句只影响其后的绘图语句，且遵循最后出现的语句有效的原则。例如，若有下述 ROT 语句序列：

ROT IS 0.7 ；

ROT IS 1.57 ；

则随后的绘图语句将按 1.57 而不是 0.7 弧度旋转。

3、无论 ORIGIN、ROT 和 SCALE 语句的出现顺序如何，图形的变换顺序总是：比例变换→旋转变换→平移变换

4、语言对大小写不敏感，例如 for、For、FOR 等，均被认为是同一个保留字。

5、语句中表达式的值均为双精度类型，旋转角度单位为弧度且为逆时针旋转，平移单位为点。

3 函数绘图程序举例

1） 基本绘图程序

```
Origin is (100,300);                    --设置原点偏移量
Rot is 0;                               --设置旋转角度（不旋转）
Scale is (1,1);                         --设置横坐标和纵坐标的比例
For T from 0 to 200 step 1 draw (t,0);
                                        --横坐标的轨迹（纵坐标为 0）
```

For T from 0 to 150 step 1 draw (0,-t);

--纵坐标的轨迹（横坐标为 0）

For T from 0 to 120 step 1 draw (t,-t);

--函数 f（t）=t 的轨迹

默认值：

Origin is (0,0)

Rot is 0;

Scale is (1,1);

2） 循环绘图（FOR-DRAW）语句

示例：FOR T FROM 0 TO 2*PI STEP PI/50 DRAW (cos(T), sin(T));

说明：该语句的作用是令 T 从 0 到 2*PI、步长 PI/50，绘制出各个点的坐标(cos(T)，sin(T))，即一个单位圆。

3） 比例设置（SCALE）语句

示例：SCALE IS (100, 100);

说明：将横坐标和纵坐标的比例设置为 1:1，且放大 100 倍。

4） 坐标平移（ORIGIN）语句

示例：ORIGIN IS (360, 240);

说明：将原点从(0, 0)平移到(360, 240) 处。

5） 角度旋转（ROT）语句

示例：将原点从(0, 0)平移到(360, 240) 处。

说明：逆时针旋转 PI/2，即逆时针旋转 90 度。

6） 注释语句

语法：// This is a comment line 或 -- 此行是注释

语义：// 或 -- 之后，直到行尾，均是注释

4 开发环境及配置

1） 开发环境

C++语言环境,本代码在 Codeblocks 上运行调试,主流 C++解释器均可运行，如 Vscode，DevC++等。

2） 配置

预设 C++函数库以及 EasyX 绘图库插件即可

5 基本原理与解决思路

1） 编写词法分析器

构造出来的词法分析器需要做到以下三个任务:

1、 过滤掉源程序中注释和无用成分（空格、TAB 等）；

2、 输出记号流以供语法分析器使用;

3、 识别非法输入，报错;

解决思路:

1、 设计记号、对函数绘图语言记号类别进行划分，用枚举类型表示。

2、 函数绘图语言的词法用正规式集合表示。

3、 预先设定区分记号的符号表，将主要内容放进符号表，词法分析器识别 ID，可通过查找符号表确定记号。

4、 得到最终简化的 DFA。

5、 分析测试。

2) 编写语法分析器

1、 规定语言的文法

简易直观的二义文法 G1，可以直接作为自下而上的语法分析器使用，但最终构造的是递归下降的语法分析器，于是就要把二义文法改写成非二义文法，消除左递归和公共左因子，形成最终文法。

2、 根据文法编写程序

编写递归下降子程序，其中主要的子程序如下：

Void Parse(char *SrcFilePtr);

Void Statement();

Void OriginStatement();

Void Rotstatement();

Void ScaleStatement();

Void ForStatement();

3、 对程序进行测试分析

3) 编写语义分析器

根据语法分析得到的语言结构中加入相应的语义动作，实现函数绘图，针对递归下降子程序而言，在各个子程序适当位置嵌入相应的语义函数。

解决方法：

1、 在上述步骤得到的递归下降子程序适当位置加入语义函数。

2、 设置多个容器来储存获取的图形参数，方便后面绘图函数的使用；

3、利用<graphics.h>库设置画图函数；

4、语义分析器测试分析调整。

6 关键类及主要方法

该部分罗列各分析器主要函数名称及作用注释，省略详细具体代码。

1） 词法分析器中 Scanner 类

```
Class Scanner {
Pubic:
    //打开文件
    Void Openfile（）；
    //关闭文件
    Void Closefile（）；
    //添加字符
    Void Add_word();
    //查找字典
    Void Search_dict();
    //生成新单词
    Void Creat_newword();
    //获取文件第一个单词
    Void Gets();
```

```
Private:
        String File,Temp = “”;
        Fstream O;
};
```

2) 语法分析器中 Parsers 类

```
class Parsers {
public:
        //生成表达式树的节点
        struct Node* Node();
        //输出
        void OutNode();
        //从记号流里获取一个记号
        void FToken();
        //从记号流里获取的记号和这个单词类型进行匹配
        void Match();
        void Error();
        //开始生成语法树
        void Parser();
        void Program();
        //处理表达式中的元素、幂运算、因子、项、表达式
        struct ExprNode* Atom()、Component()、Factor()、Term()、
       Expression();
```

3)　图形实现代码

1、 坐标系上绘制一个点 void Draw_graph();

2、 循环绘制曲线点形成曲线图 void DrawCircle ();

7 测试截图

测试文本如下

data.txt

文件　编辑　查看

```
origin is (350,220);

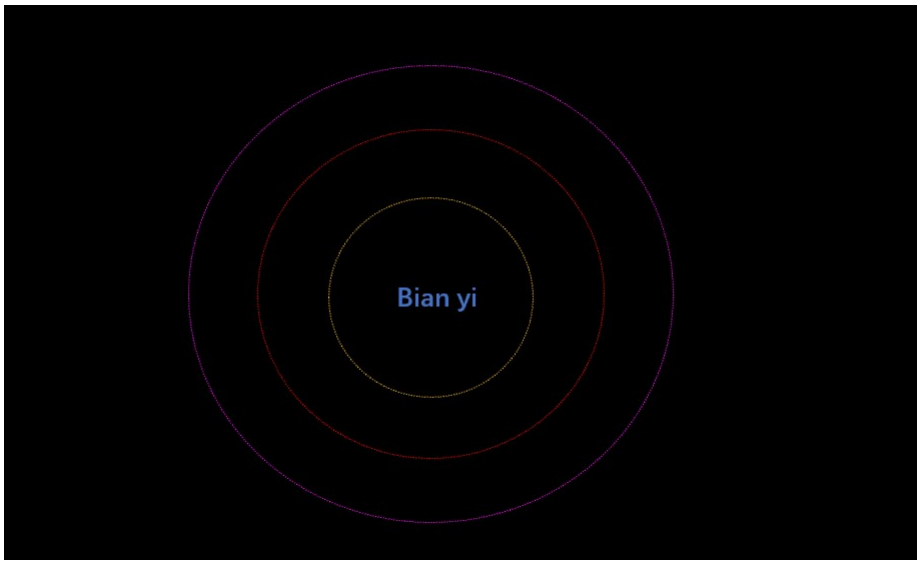
scale is (50,50);
color is (255,255,0);
for t from 0 to 2*pi step pi/100 draw(cos(t),sin(t));  --内环黄色的圆圈

scale is (50,50);
color is (255,0,0);
for t from 0 to 2*pi step pi/100 draw(cos(t),sin(t));  --中间红色的圆圈

scale is (50,50);
color is (255,0,255);
for t from 0 to 2*pi step pi/100 draw(cos(t),sin(t));  --外环紫色的圆圈

color is (0,0,255);
notes:(310,220,"bian yi"); --中心文本显示蓝色的bian yi
```

最终程序输入并运行图形结果如下：



8 总体体会（遇到的问题与体会）

1） 问题

1、 最初，我对课堂上学到的词法分析器、语法分析器和语义分析器的理解并不深入，这导致我在实验开始时缺乏清晰的思路，不知道如何着手。通过查阅资料和不断学习，我逐渐找到了解决问题的方法。

2、 考虑到需要进行图形绘制，我选择了 C++语言，并利用其内置的库。尽管 C++与 C 语言有许多相似之处，但我意识到我需要更深入地学习 C++，以便更好地掌握其特性和库的使用。

3、 我对文件的输入输出操作掌握得不是很好，这在实验中造成了一定的困扰。为了克服这个问题，我参考了他人的代码，逐步学习和掌握了文件操作的相关技巧。

4、 在整体代码的调试过程中，我遇到了许多错误和问题。我通过查看错误信息，查找相关资料，逐步解决了这些问题，这个过程中我学到了很多宝贵的知识。

2） 体会

1、 我深刻体会到，课本上的知识需要通过实践来加深印象和理解。只有将知识融会贯通，才能有效地将理论应用于实际问题的解决中。

2、 我也认识到，代码能力的提升需要大量的实践。单纯的思考是不够的，只有通过不断的编码实践，才能提

高编程技能。

3、 本次实验让我对C++中平时接触较少的部分，如图形绘制和文件读取操作，有了更深入的理解。这将有助于我在未来更好地应用这些知识。

4、 实验过程中，我增强了解决问题的能力，学会了如何有效地使用搜索引擎来查找资料和解决方案。同时，我也加深了对编译器工作原理的理解，这些收获对我未来的学习和工作都将大有裨益。